Practical 10: Presenting Data

Visualisation and Tables

Table of contents

1 Preamble 1

2 Where We Left Off 3

3 Using Matplotlib 4

4 Create an Atlas 7

5 Think Text! 11

6 Using Bokeh 11

7 Appendix: Finding Fonts 14

> **Important**
>
> This practical focusses on the final topic we want to cover in *Foundations*: visualisation! You will have seen and learned quite a bit of this across the preceding weeks, but it was done in an *ad-hoc* way, here we try to systematise things a bit. We pick up directly from Practical 9: the data you linked and saved there is what we are going to present.

> **Connections**
>
> Here we're trying to tidy up the loose ends. You've already worked with basic data visualisations in Seaborn and Matplotlib (including (geo)panda's `plot` function), but we want you to have a better sense of how that *works* as part of a coherent – if altogether rather complex and overwhelming – approach to managing a data visualisation. We'll finish by automating the whole process so that a single function produces a consistent 'atlas' page for *every* London borough.

1 Preamble

```
import geopandas as gpd

from pathlib import Path
```

```
import matplotlib.pyplot as plt
from matplotlib import font_manager
```

```
from pathlib import Path
from requests import get
from functools import wraps

def check_cache(f):
    @wraps(f)
    def wrapper(src:str, dst:str, min_size=100) -> Path:
        if src.find('?') == -1:
            url = Path(src)
        else:
            url = Path(src[:src.find('?')])
        fn  = url.name  # Extract the filename
        dsn = Path(f"{dst}/{fn}") # Destination filename
        if dsn.is_file() and dsn.stat().st_size > min_size:
            print(f"+ {dsn} found locally!")
            return(dsn)
        else:
            print(f"+ {dsn} not found, downloading!")
            return(f(src, dsn))
    return wrapper

@check_cache
def cache_data(src:Path, dst:Path) -> str:
    """Downloads a remote file.

    The function sits between the 'read' step of a pandas or geopandas
    data frame and downloading the file from a remote location. The idea
    is that it will save it locally so that you don't need to remember to
    do so yourself. Subsequent re-reads of the file will return instantly
    rather than downloading the entire file for a second or n-th itme.

    Parameters
    ----------
    src : str
        The remote *source* for the file, any valid URL should work.
    dst : str
        The *destination* location to save the downloaded file.

    Returns
    -------
    str
        A string representing the local location of the file.
    """

    # Create any missing directories in dest(ination) path
    # -- os.path.join is the reverse of split (as you saw above)
    # but it doesn't work with lists... so I had to google how
```

```
    # to use the 'splat' operator! os.makedirs creates missing
    # directories in a path automatically.
    if not dst.parent.exists():
        dst.parent.mkdir(parents=True, exist_ok=True)

    # Download and write the file
    with dst.open(mode='wb') as file:
        response = get(src)
        file.write(response.content)

    print(' + Done downloading...')

    return dst.resolve()
```

2 Where We Left Off

Practical 9 finished by saving three files to `data/geo`. If one of them is missing then the cell below will tell you so, and you'll need to go back to Practical 9 and run it from the top.

```
gdir = Path('data/geo')

# This must match the LA set in the Preamble of Practical 9
LA      = 'Waltham Forest'
LA_slug = LA.lower().replace(' ','-')

files = {
    'listings': gdir / 'Listings_with_LA.geoparquet',
    'msoas':    gdir / 'London_MSOA_Names.geoparquet',
    'la_msoas': gdir / f'{LA_slug}-MSOA_data.geoparquet',
}
missing = [str(p) for p in files.values() if not p.exists()]
if missing:
    raise FileNotFoundError(f"Run Practical 9 first; missing: {', '.join(missing)}")

# Only load the columns we need: the listings file in particular is large
listings = gpd.read_parquet(files['listings'], columns=['listing_url','room_type','p
msoas    = gpd.read_parquet(files['msoas'], columns=['MSOA11CD','MSOA11NM','Borough'
msoa_gdf = gpd.read_parquet(files['la_msoas'])

print(f"{LA}: {msoa_gdf.shape[0]} MSOAs; London: {msoas.shape[0]} MSOAs and {listing
```

```
Waltham Forest: 28 MSOAs; London: 983 MSOAs and 85,816 listings.
```

3 Using Matplotlib

3.1 Anatomy of a Figure

> **Tip**
>
> You might want to bookmark the 'Anatomy of a Figure' image so that you can easily find and refer to it in the future. This structure is why `matplotlib` is so much nastier than `ggplot`, but it does also give you greater *control* over the output if you really dig into the guts of things.

One of the reasons that Matplotlib is so much more complex than `ggplot` is that it can actually *do* a lot more than ggplot, including image manipulation, axis translation, and even 3D. You can get a sense of this by looking at the tutorials since the Users guide can be a bit overwhelming.

Nevertheless, the core components of all matplotlib figures can be seen here:

3.2 Fonts

I find matplotlib's use of fonts to be *profoundly* weird. The format for specifying fonts on a per-figure basis is a dictionary (a 'fontdict'), so where you see `fontdict` in the `matplotlib` documentation the following should work:

```
font = {'family': 'serif',
        'color':  'darkred',
        'weight': 'normal',
        'size': 16,
        }
```

> **Warning**
>
> Depending on how you are running the notebook, you may not have access to all of the fonts on your computer: if the notebook is running in a Podman container it only has the fonts that are installed in the container. In other words: **don't panic if you don't see the font(s) you want.** The code below picks the first font from a list that is actually available (see the appendix if you want to explore what is installed).

We are going to set a 'title font' (`tfont`), a 'body copy font' (`bfont`), and an 'axis font' (`afont`) to re-use across multiple plots to ensure consistency of output. You can pick another font and see what happens.

```
available = {f.name for f in font_manager.fontManager.ttflist}
ff = next((f for f in ['Liberation Sans','Helvetica','Arial'] if f in available), 'D
print(f"Using the {ff} font.")

tfont = {'fontname':ff}
bfont = {'fontname':ff, 'weight':'bold', 'horizontalalignment':'left'}
afont = {'fontname':ff}
```

```
Using the Liberation Sans font.
```

3.3 The Defaults

At this point we're going to work towards a kind of 'atlas' that would make it easy to compare some features for different London boroughs (I basically implemented a the basic `matplotlib` version of QGIS' Atlas functionality). First, we pull out the two columns of the data frame from Practical 9 that we want to map:

```
median_gdf  = msoa_gdf[['MSOA11CD','median_price','geometry']]
listing_gdf = msoa_gdf[['MSOA11CD','listing_count','geometry']]
```

Here is a demonstration of some of the ways you can adjust features in a Python matplotlib plot. I'm not suggesting either of these is a *good* output, but that's not the point! The idea is to see the various ways you can tweak a plot… And notice that we've not yet changed any fonts. And it shows.

```
# Set up a 1 x 2 plot (you can also leave off the nrows= and ncols=)
f,axes = plt.subplots(nrows=1, ncols=2, figsize=(8,6))
# ax1 will be the first plot on the left, ax2 will be on the right;
# a 2 (or more) *row* plot will return a list of lists... 1 list/row.
ax1 = axes[0]
ax2 = axes[1]

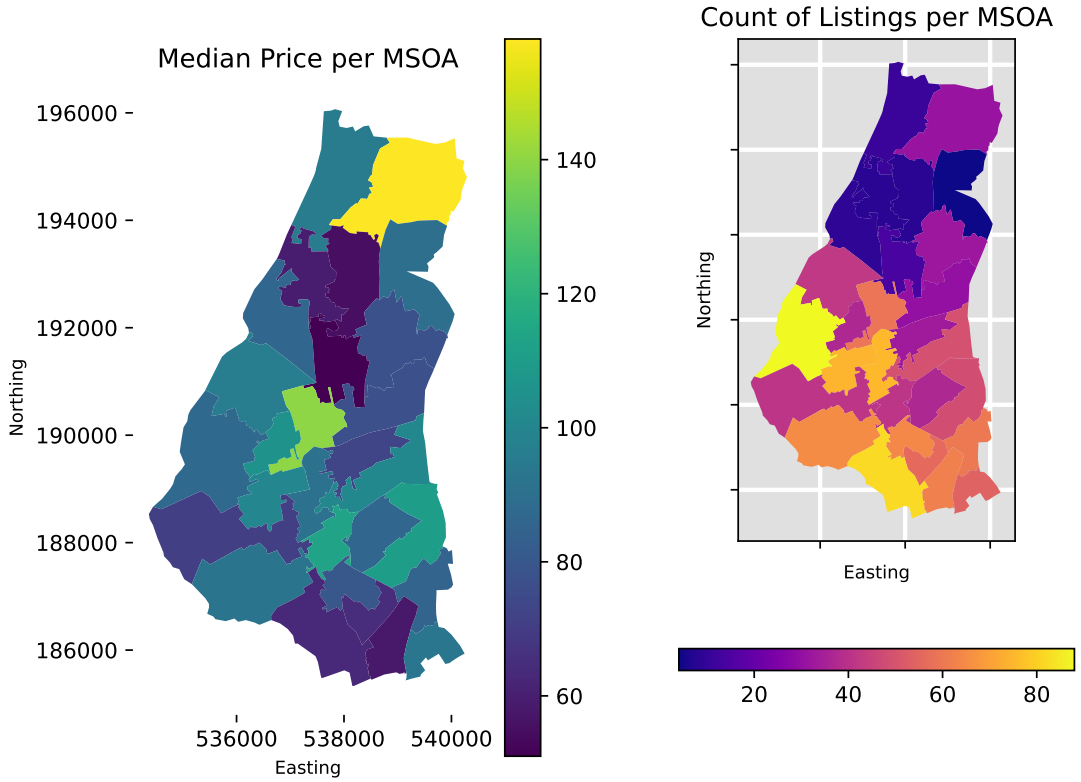
# Left plot is the median price
median_gdf.plot(column='median_price', ax=ax1, legend=True, cmap='viridis')
ax1.set_title("Median Price per MSOA");
# Turn off the frame, one side of the plot at a time
ax1.spines['top'].set_visible(False)
ax1.spines['right'].set_visible(False)
ax1.spines['bottom'].set_visible(False)
ax1.spines['left'].set_visible(False)
# Set the labels
ax1.set_xlabel("Easting");
ax1.set_ylabel("Northing");

# Right plot is the number of listings; note
# here the use of both zorder (which is the
# 'stacking order' of elements on the plot, and
# the legend_kwds (keywords) to change the
# orientation of the plot to horizontal
listing_gdf.plot(column='listing_count', ax=ax2, legend=True, cmap='plasma', zorder=
                 legend_kwds={"orientation": "horizontal"})
ax2.set_title("Count of Listings per MSOA");
# Set a background colour for the plot
ax2.set_facecolor((.4, .4, .4, .2))
# Add grid lines and set their zorder to
# below that of the data on the plot
ax2.grid(visible=True, which='major', axis='both', color='w', linestyle='-', linewid
ax2.set_axisbelow(True)

# Remove the labels on the ticks of the
# axes (meaning: remove the numbers on
```

```
# x- and y-axes).
ax2.set_xticklabels([])
ax2.set_yticklabels([])

# Set the labels
ax2.set_xlabel("Easting");
ax2.set_ylabel("Northing");
```



3.4 Improving on Defaults

Now we apply the fonts. Notice the `**tfont` below: this is a way of expanding the `tfont` dictionary into a set of keyword parameters to a function call. We do the same for the titles on each map, but passing a different fontdict.

Question

```
f,axes = plt.subplots(1,2,figsize=(8,6))

median_gdf.plot(column='median_price', ax=axes[0], legend=True, cmap='viridis')
listing_gdf.plot(column='listing_count', ax=axes[1], legend=True, cmap='plasma')
for ax in axes:
    ax.???('off')

# The 'super-title' is no longer centred (x=0.025) but left-aligned
f.suptitle(LA, x=0.025, ha='left', size=24, **???)
axes[0].set_title('Median Price', size=20, **???)
```

```
axes[1].set_title('Count', size=20, **???)

# And add a short piece of text below the borough
plt.figtext(x=0.88, y=0.957, linespacing=1.4, va='top', ha='right', size=12,
            s=f"Total listings: {listing_gdf['listing_count'].sum():,.0f}  Median pr
f
```

4 Create an Atlas

4.1 Adding Picture-in-Picture

We're now going to emulate a *bit* of QGIS' Atlas function by creating two subplots and then adding a *third* plot afterwards that shows where the borough is.

```
spath = 'https://github.com/jreades/fsds/blob/master/data/src/' # source path
boros = gpd.read_file( cache_data(spath+'Boroughs.gpkg?raw=true', gdir) )
```

```
+ data/geo/Boroughs.gpkg found locally!
```

Question

```
f,axes = plt.subplots(1,3,gridspec_kw={'width_ratios':[1,4,4]}, figsize=(8,6))

# Plot 0 is basically being used as a 'spacer'
axes[0].axis('off')

median_gdf.plot(column='median_price', ax=axes[1], legend=True, cmap='viridis')
axes[1].set_title('Median Price', size=20, **afont)
listing_gdf.plot(column='listing_count', ax=axes[2], legend=True, cmap='plasma')
axes[2].set_title('Count', size=20, **afont)

for ax in axes[1:]:
    ax.axis('off')

# Add a *third* chart that we use as a kind of 'PiP' to show which borough
# we're talking about. The add_axes call takes the [left, bottom, width,
# height] of the new chart as fractions of the whole figure.
ax3 = f.add_axes([0.015, 0.7, 0.2, 0.2])
boros.plot(facecolor='lightgrey', edgecolor='w', linewidth=1, ax=ax3)
boros[boros.NAME==???].plot(facecolor='r', edgecolor='none', hatch='///', ax=ax3)
ax3.axis('off')

f.suptitle(LA, x=0.025, ha='left', size=24, **tfont)
plt.figtext(x=0.025, y=0.65, s=f"Total listings: {listing_gdf['listing_count'].sum()
```

4.2 Data for Every Borough

An atlas needs a page for *every* borough, and each page needs its own data. So we'll go back to the two London-wide files from Practical 9 and do for all of London what we did for one borough there: join each listing to its MSOA (a spatial join), then aggregate the listings by MSOA.

```
joined = gpd.sjoin(listings[['price','geometry']], msoas[['MSOA11CD','geometry']], p

agg = joined.groupby('MSOA11CD').agg(
    listing_count = ('price','count'),
    median_price  = ('price','median')
).reset_index()

# Left join so that MSOAs without any listings are kept (as zeros/NaN)
london = msoas[['MSOA11CD','MSOA11NM','Borough','Subregion','geometry']].merge(agg,
london['listing_count'] = london.listing_count.fillna(0)

print(f"{london.shape[0]:,} MSOAs in {london.Borough.nunique()} boroughs.")
london.head(3)
```

```
983 MSOAs in 33 boroughs.
```

	MSOA11CD	MSOA11NM	Borough	Subregion	geo
0	E02000001	City of London 001	City of London	Outer West and North West	MU
1	E02000002	Barking and Dagenham 001	Barking and Dagenham	Outer East and North East	MU
2	E02000003	Barking and Dagenham 002	Barking and Dagenham	Outer East and North East	MU

> **Why not use the borough in the join?**
>
> We join on the `MSOA11CD` and not on the borough because an MSOA is the unit that we want to *map*. The borough is then just a way of splitting the result up into pages.

Now split the result up by borough and save one file for each. The file names follow the same pattern as Practical 9 (`<borough-slug>-MSOA_data.geoparquet`) so that you can find them again.

Question

```
atlas_dir = gdir / 'atlas'
atlas_dir.mkdir(parents=True, exist_ok=True)

for borough, gdf in london.groupby(???):
    slug = borough.lower().replace(' ','-')
    gdf.to_parquet(???)

print(f"Saved {len(list(atlas_dir.glob('*.geoparquet')))} files.")
```

4.3 A Function for One Page

If you were doing this a lot it should be a function, and there is one more design decision to make first: **consistent colours**. If each borough chooses its own colour range then bright yellow means one thing in Waltham Forest and something quite different in Westminster, so we work out the minimum and maximum for *London* once and pass them to every page.

```
scale = {
    'median_price':  (london.median_price.min(), london.median_price.max()),
    'listing_count': (0, london.listing_count.max()),
}
scale
```

```
{'median_price': (np.float64(27.0), np.float64(519.0)),
 'listing_count': (0, np.int64(1038))}
```

Question

```
def atlas_page(la:str, boros:gpd.GeoDataFrame, scale:dict, outdir:Path):
    slug = la.lower().replace(' ','-')
    gdf  = gpd.read_parquet(atlas_dir / f'{slug}-MSOA_data.geoparquet')

    f,axes = plt.subplots(1,3,gridspec_kw={'width_ratios':[1,4,4]}, figsize=(8,6))
    axes[0].axis('off')

    for ax, col, cmap, title in zip(axes[1:], ['median_price','listing_count'],
                                    ['viridis','plasma'], ['Median Price','Count']):
        vmin, vmax = scale[???]
        gdf.plot(column=col, ax=ax, legend=True, cmap=cmap, vmin=vmin, vmax=vmax)
        ax.set_title(title, size=20, **afont)
        ax.axis('off')

    ax3 = f.add_axes([0.015, 0.7, 0.2, 0.2])
    boros.plot(facecolor='lightgrey', edgecolor='w', linewidth=1, ax=ax3)
    boros[boros.NAME==???].plot(facecolor='r', edgecolor='none', hatch='///', ax=ax3
    ax3.axis('off')

    f.suptitle(la, x=0.025, ha='left', size=24, **tfont)
    plt.figtext(x=0.025, y=0.65, s=f"Total listings: {gdf['listing_count'].sum():,.0

    f.savefig(outdir / f'{slug}.png', dpi=150, bbox_inches='tight')
    plt.close(f) # Otherwise 33 figures stay open in memory
```

4.4 Automate All the Things

Now the function does the work. Test it on one borough first and then loop over the rest:

Question

```
img_dir = Path('data/atlas_img')
img_dir.mkdir(parents=True, exist_ok=True)

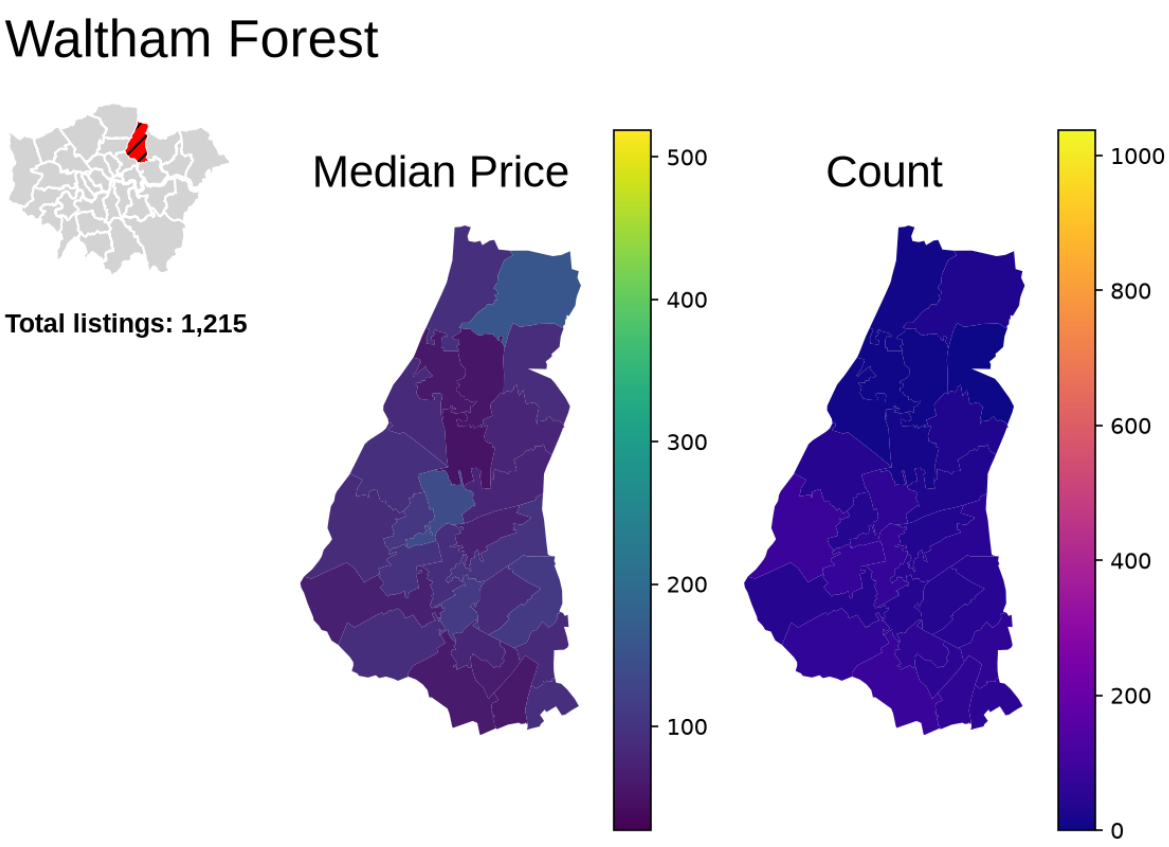
atlas_page(LA, boros, scale, img_dir)

for la in sorted(london.???.unique()):
    atlas_page(la, boros, scale, img_dir)

print(f"Saved {len(list(img_dir.glob('*.png')))} pages to {img_dir}.")
```

Here is one of the pages (open the others from `data/atlas_img`):

```
from IPython.display import Image
Image(img_dir / f'{LA_slug}.png', width=600)
```



4.5 Bonus Achievement Unlocked!

If you have the time and inclination, here are some ways to improve the atlas:

1. The placement of the picture-in-picture (and the title) needs a rethink for boroughs with long names, such as ‘Barking and Dagenham’ or ‘Kensington and Chelsea’.
2. Median price is very skewed by a few expensive MSOAs. How would a different colour scale (quantiles? a log scale?) change the pages? Remember that it still needs to be the *same* scale for every borough.
3. MSOAs with no listings are currently drawn as empty. Could you mark them in a way that explains why they are empty?

4. Add some more text to each page: maybe pull some content from Wikipedia using Beautiful Soup (`bs4`)?
5. Combine the pages into a single PDF.

5 Think Text!

I also wanted to draw your attention to this outstanding piece on using text effectively in data visualisation: we often add labels as afterthoughts without too much regard for where they go or how they look; however, getting the content, positioning, size, and even font/font-weight 'right' can make all the difference to the effectiveness of your chart! The illustrations are top-notch.

And see the bibliography at the end!

> Bookmark Me!
>
> Basically, bookmark this blog post and refer to it every time you are making a map or chart.

6 Using Bokeh

Bokeh can do a *lot* more than this, but I just wanted to give you a flavour of the other visualisation tools supported by Python. This obviously works *very* differently in setup and use: instead of a static image we get an interactive chart that you can hover over, pan, and zoom.

```
from bokeh.io import output_notebook, show
from bokeh.plotting import figure
from bokeh.models import ColumnDataSource, GeoJSONDataSource, LinearColorMapper, Col
from bokeh.palettes import brewer

output_notebook()
```

```
Unable to display output for mime type(s): text/html
```

```
Unable to display output for mime type(s): application/javascript, application/vnd.boke
```

6.1 For a Chart

Group the listings by Room Type, and aggregate by median price, also producing a count variable for the number of listings of each type in the borough.

```
la_tots = listings[listings.NAME==LA].groupby(by='room_type', observed=False).agg(
                            {'price':'median', 'listing_url':'count'}
                        ).reset_index().rename(columns={'listing_url':'count'})
la_tots
```

	room_type	price	count
0	Entire home/apt	127.0	980
1	Hotel room	NaN	0
2	Private room	50.0	733
3	Shared room	40.0	1

```
room_types = la_tots.room_type.to_list()

# Add hover tooltip
source = ColumnDataSource(data=dict(
    rt    = room_types,
    count = la_tots['count'].to_list(),
    price = la_tots.price.to_list(),
))

TOOLTIPS = [
    ("Room Type", "@rt"),
    ("Number of Listings", "@count{,}"),
    ("Median Price", "$@price{,}/night")
]

p = figure(x_range=room_types, height=300, tooltips=TOOLTIPS,
           title=f"Median Price by Room Type in {LA}",
           toolbar_location=None, tools="")

p.vbar(x='rt', top='price', width=0.9, source=source)
p.xgrid.grid_line_color = None
p.y_range.start = 0

show(p)
```

```
Unable to display output for mime type(s): text/html
```

```
Unable to display output for mime type(s): application/javascript, application/vnd.boke
```

6.2 For a Map

This is not the prettiest code, but it should work… Bokeh wants GeoJSON, so the function below converts a GeoDataFrame and then builds the map. It was cobbled together from two tutorials (this one and this one). As you can see, this is a very different approach to mapping data, but it has clear benefits for exploratory purposes and produces fast, interactive maps… and I've not even added selection and filtering tools!

```
import json

def get_geodatasource(gdf):
    """Get a GeoJSONDataSource from a GeoPandas object"""
    return GeoJSONDataSource(geojson=json.dumps(json.loads(gdf.to_json())))
```

```
def bokeh_plot_map(gdf, column, title=''):
    """Plot a Bokeh map of one column of a GeoDataFrame"""

    geosource = get_geodatasource(gdf)
    palette   = brewer['OrRd'][8][::-1]
    vals      = gdf[column]

    # LinearColorMapper maps numbers in a range onto a sequence of colours
    color_mapper = LinearColorMapper(palette=palette, low=vals.min(), high=vals.max(
    color_bar    = ColorBar(color_mapper=color_mapper, label_standoff=8, width=500,
                            location=(0,0), orientation='horizontal')

    p = figure(title=title, height=700, width=850, toolbar_location='right',
               tools='wheel_zoom,pan,reset,hover')
    p.add_tile("CartoDB Positron", retina=True)
    p.xgrid.grid_line_color = None
    p.ygrid.grid_line_color = None

    # Add patch renderer to figure
    p.patches('xs','ys', source=geosource, fill_alpha=0.5, line_width=0.5, line_colo
              fill_color={'field':column, 'transform':color_mapper})
    p.add_layout(color_bar, 'below')

    # Add hover
    hover = p.select_one(HoverTool)
    hover.point_policy = "follow_mouse"
    hover.tooltips = [("Borough", "@Borough"),
                      ("MSOA", "@MSOA11NM"),
                      ("Count of Listings", "@listing_count"),
                      ("Median Price", "$@median_price")]
    return p
```

The tile map that sits underneath the data is in Web Mercator (EPSG:3857), so we need to reproject the data first. And that's where the per-borough data from above comes in handy:

```
la_msoas = london[london.Borough==LA].to_crs('epsg:3857')
la_msoas.total_bounds
```

```
array([-6.92259597e+03,  6.71915726e+06,  2.86516058e+03,  6.73646377e+06])
```

And map it!

```
show(bokeh_plot_map(la_msoas, 'median_price', title=f'MSOA-Level Activity in {LA}'))
```

```
/opt/conda/lib/python3.13/site-packages/bokeh/models/plots.py:439: UserWarning: CartoDB
  url = selected_provider.build_url(scale_factor=scale_factor)
```

```
Unable to display output for mime type(s): text/html
```

```
Unable to display output for mime type(s): application/javascript, application/vnd.boke
```

> ℹ Connections
>
> And that's it. That's all she wrote! You've now covered in <10 weeks what many people might take 10 *months* to cover. So do not feel like either: 1) you know it all; or 2) you know nothing. You have learned a *lot*, but it's probably just enough to see how much you *don't* know. And *that* is the start of wisdom. Good luck, young Python-master!

7 Appendix: Finding Fonts

If you use `conda` and install directly on to the computer then you *might* have access to all of your computer's fonts (though there are different *types* of fonts as well, not all of which will show up), but for most users it will be those that were installed into Podman. Here are some ways to see what you have.

7.1 Listing Fonts in Matplotlib

```
from IPython.display import HTML

names = sorted({f.name for f in font_manager.fontManager.ttflist if not f.name.start
print(f"Found {len(names)} fonts.")

code = "\n".join([f"<p><span style='font-family: {name}; font-size: 18px;'>{name}</s
HTML("<div style='column-count: 2;'>{}</div>".format(code))
```

```
Found 204 fonts.
```

```
<IPython.core.display.HTML object>
```

7.2 Using Fontconfig

`fontconfig` is the base Linux utility for managing fonts. We can list fonts using `fc-list` and then a set of 'switches' determining the kind of information we want back. Since fontconfig doesn't exist on OSX or Windows you may not be able to run these.

Here we ask fontconfig to format the output so that we only get the first part of the family name, and then we pipe (recall `|` sends output from one utility to another!) the output of that to `sort`, which sorts the output, and `uniq` which removes duplicates (which there will be because there are **bold**, *italic*, small-caps, etc. versions of each font). To make better sense of this you can always try playing around with all three steps!

```
fonts = ! fc-list --format="%{family[0]}\n" | sort | uniq
print(fonts[:5])
```

> **Capturing output**
>
> Notice that we're able to capture the output of an external application (called via the Terminal) with `fonts = ! ...`. This can be useful when something is easy to do on the command line but hard to do in Python.

The next option also pipes output from fontconfig, but to the `grep` utility which checks each line for the character sequence `Liberation`. Now we're asking fontconfig to include `style` details which will relate to both weight (regular, bold, extra bold, light, etc.) and italic, bold, small caps, etc. Then we ask for the languages supported by one font, and the monospace fonts installed:

```
fonts = ! fc-list : family style | grep "Liberation"
print(sorted(fonts)[:5])

langs = ! fc-list "Liberation Mono" : lang
print(sorted(langs)[:5], "...")

monos = ! fc-list :spacing=mono : family | sort | uniq
print(sorted(monos)[:5], "...")
```

You can find more examples here, a more detailed set of instructions here, and even information about (for example) supported languages based on RFC 3066.